Programación Orientada a Objetos
Boletín 4

El objetivo de este boletín es estudiar el uso de interfaces y su relación con los mecanismos de herencia entre clases, así como el tratamiento de excepciones que tienen lugar durante la ejecución de un programa. Más concretamente:

- Creación de interfaces y jerarquías de interfaces.
- Creación de métodos estáticos, métodos por defecto y constantes en los interfaces.
- Creación y gestión de excepciones.

Ejercicio 1

1. Crear un proyecto denominado `B4E1` en el que se crearán los interfaces correspondientes a las clases `Pelicula`, `Actor` y `Director`; y la clase `PrincipalB4E1` en la que se crearán instancias/objetos de para comprobar que los interfaces funcionan correctamente y que se logra la independencia de las clases.

2. Crear los siguientes interfaces:

- **Nombre:** `ICPelicula`
  Métodos: los getters y los setters que han sido implementados en la clase `Pelicula`.
- **Nombre del interface:** `IAPelicula`
  Métodos: `darAlta`, `actoresDirigidos` y `actoresFracaso`.
- **Nombre del interface:** `ITPelicula`
  Métodos: `recaudacionTotal`, `presupuesto` y `exito`.
- **Nombre del interface:** `IPelicula`
  Restricción: debe heredar de los interfaces `IAPelicula`, `ICPelicula` e `ITPelicula`.
  Método por defecto:
  - `public default int numeroActores()`: devuelve el número de actores que han participado en la película.
  - `public default float costeActores()`: devuelve el coste de los actores que han participado en la película.

La clase `Pelicula` deberá implementar el interface `IPelicula`.

3. Crear los siguientes interfaces:

- **Nombre:** `IParticipante`
  Métodos: el método `introducirPelicula`, tal y como ha sido definido tanto en las clases `Actor` y `Director`.
- **Nombre:** `IActor`
  Métodos: los getters y los setters que han sido implementados en la clase `Actor`, y los métodos `peliculasDeGenero`, `peliculasFracaso`, `importancia`, `totalGancias`, `deseosNoCumplidos`.
  Métodos estáticos:

- `public boolean participa(String nombre, Pelicula pelicula)`: devuelve si un actor con el nombre `nombre` ha participado en `pelicula`.

Restricción: debe heredar del interface `IParticipante`.

- **Nombre:** `IDirector`
  Métodos: los getters y los setters que han sido implementados en la clase `Director`, así como los métodos `dirigirActor`, `generoMasCaro` y `peliculaMasBeneficio`.
  Restricción: debe heredar del interface `IParticipante`.

La clase `Director` deberá implementar el interface `IDirector`, mientras que clase `Actor` deberá implementar el interface `IActor`.

4. Modificar las clases `Pelicula`, `Actor` y `Director` para que funcionen únicamente con los interfaces definidos en el **apartado 3**, lo que requerirá también la modificación de los métodos de los propios interfaces. Por ejemplo, para introducir una película en la clase Actor, el método debería ser el siguiente: `public void introducirPelicula(IPelicula película)`, donde se puede ver que el argumento es un `IPelicula` en vez de una `Pelicula`.

Nota: si a la hora de realizar el ejercicio 4 es necesario introducir algún método adicional en los interfaces, se deberá indicar como un comentario en el código de la clase `Pelicula`, `Actor` o `Director`, según corresponda. En ese comentario se explicará cuál ha sido esa modificación y por qué ha sido necesario llevarla a cabo.

5. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: `PrincipalB4E1`

Operaciones a realizar en la función `main()`:

a. Crear las instancias de los actores `actor1`, `actor2`, `actor3`, `actor4`, `actor5` y `actor6`.

b. Mostrar las características de los actos creados en (**a**).

c. Crear las instancias de las películas `pelicula1`, `pelicula2`, `pelicula3` y `pelicula4`.

d. Mostrar por pantalla las características de las películas `pelicula1`, `pelicula2`, `pelicula3` y `pelicula4`.

e. Crear las instancias de los directores asociados a las películas generadas en el apartado (**c**).

f. Mostrar por pantalla las características de los directores creados en el apartado (**e**).

g. Mostrar el número de actores para `pelicula1` y `pelicula2` usando el correspondiente método por defecto del interface `IPelicula`.

h. Mostrar el coste de los actores para `pelicula3` y `pelicula4` usando el correspondiente método por defecto del interface `IPelicula`.

i. Indicar si "Leonardo DiCaprio" ha participado en la película `pelicula2` usando el método estático del interface `IActor`.

j. Probar todos los métodos de `Pelicula` para la instancia `pelicula3` (salvo `equals` y `toString`).

k. Probar todos los métodos de `Actor` para la instancia `actor2` (salvo `equals` y `toString`).

l. Probar todos los métodos de `Director` para el director de la `pelicula4` (salvo `equals` y `toString`).

Ejercicio 2

1. Crear un proyecto denominado `B4E2` en el que se crearán clases para las excepciones correspondientes a los errores que pueden tener lugar en tiempo de ejecución. Concretamente, se crearán excepciones que están asociadas a las clases `Pelicula`, `Actor` y `Director`. También se creará la clase `PrincipalB4E2` en la que se crearán instancias/objetos de para comprobar que las excepciones funcionan correctamente y que se capturan los errores.

2. Crear la siguiente jerarquía de excepciones:

- `ExcepcionCine`.
- `ExcepcionPelicula`, que es una subclase de `ExcepcionCine`.
- `ExcepcionParticipante`, que es una subclase de `ExcepcionCine`.
- `ExcepcionActor`, que es una subclase de `ExcepcionParticipante`.
- `ExcepcionDirector`, que es una subclase de `ExcepcionParticipante`.

3. **Todas las excepciones se deberán lanzar en los métodos y/o constructores correspondientes y se deberán capturar en la clase principal**.

4. Lanzar la excepción `ExcepcionPelicula` para los métodos y atributos de la clase `Pelicula` cuando se den las siguientes condiciones:

- Si el argumento `nombre` del constructor de la clase `Pelicula` es `null`, cuando se capture la excepción se deberá imprimir el siguiente mensaje: "No se puede crear la película: el nombre de la película no puede ser null".
- Si el argumento `anho` del constructor o del setter del atributo `anho` de la clase `Pelicula` no está en el rango 1895 y 2022, cuando se capture la excepción se deberá mostrar el siguiente mensaje:
  - Si se captura en el constructor: "No se puede crear la película: <anho> no está en el rango de 1895 y 2021".
  - Si se captura en el setter del atributo `anho`: "No se puede modificar el año de la película <nombre>: <anho> no está en el rango de 1895 y 2021".

  donde <anho> es el valor del año que ha provocado que se haya lanzado la excepción y <nombre> es el nombre de la película.
- Si el argumento `duracion` del constructor o del setter del atributo `duracion` de la clase `Pelicula` no es mayor de 5, cuando se capture la excepción se deberá imprimir el siguiente mensaje:
  - Si se captura en el constructor: "No se puede crear la película: <duracion> no es mayor de 5".
  - Si se captura en el setter del atributo `duracion`: "No se puede modificar la duración de la película <nombre>: <duracion> no es mayor de 5".

  donde <duracion> es el valor de la duración que ha provocado que se haya lanzado la excepción y <nombre> es el nombre de la película.

5. Lanzar la excepción `ExcepcionParticipante` para los métodos y atributos de la clase `Actor` y `Director`, según corresponda, cuando se den las siguientes condiciones:

   - Si el argumento `nombre` del constructor de la clase `Actor` es `null`, cuando se capture la excepción se deberá imprimir el siguiente mensaje: "No se puede crear el actor: el nombre del actor no puede ser null".
   - Si el argumento `nombre` del constructor de la clase `Director` es `null`, cuando se capture la excepción se deberá imprimir el siguiente mensaje: "No se puede crear el director: el nombre del director no puede ser null".

6. Lanzar la excepción `ExcepcionActor` para los métodos y atributos de la clase `Actor` cuando se den las siguientes condiciones:

   - Si el argumento `cache` del constructor o del setter del atributo `cache` de la clase `Actor` no es mayor de 0, cuando se capture la excepción se deberá imprimir el siguiente mensaje:
     - Si se captura en el constructor: "No se puede crear el actor: <cache> no es mayor de 0".
     - Si se captura en el setter del atributo `cache`: "No se puede modificar el cache del actor <nombre>: <cache> no es mayor de 0".

     donde <cache> es el valor de la duración que ha provocado que se haya lanzado la excepción y <nombre> es el nombre del actor.
   - Si el argumento `deseos` del constructor o del setter del atributo `deseos` de la clase `Actor` es `null` o está vacío, cuando se capture la excepción se deberá imprimir el siguiente mensaje:
     - Si se captura en el constructor: "No se puede crear el actor: deseos no puede ser null" o "No se puede crear el actor: deseos es una lista vacía", según corresponda.
     - Si se captura en el setter del atributo `deseos`: "No se pueden modificar los deseos del actor <nombre>: la instancia de deseos es null" o "No se pueden modificar los deseos del actor <nombre>: la instancia de deseos es una lista vacía", según corresponda.

     donde <deseos> es la instancia de los deseos que ha provocado que se haya lanzado la excepción y <nombre> es el nombre del actor.
   - Si el argumento del método `importancia` no tiene un tipo de participación correcto, cuando se capture la excepción se deberá imprimir el siguiente mensaje: "El valor del argumento del método importancia no es correcto: debe ser 'principal' o 'secundario' ".

7. Lanzar la excepción `ExcepcionDirector` para los métodos y atributos de la clase `Director` cuando se den las siguientes condiciones:

   - Si el argumento `especialista` del constructor o del setter del atributo `especialista` de la clase `Director` es `null`, está vacío o tiene géneros incorrectos, cuando se capture la excepción se deberá imprimir el siguiente mensaje:
     - Si se captura en el constructor: "No se puede crear el director: <especialista> no puede ser null" o " No se puede crear el director: especialista es una lista vacía" o "No se puede crear el director: alguno de los géneros indicados en la lista no está permitido", según corresponda.
     - Si se captura en el setter del atributo `especialista`: "No se pueden modificar el atributo especialista del director <nombre>: la instancia de especialista no puede ser null" o " No se pueden modificar el atributo especialista del director <nombre>: la instancia de especialista

es una lista vacía" o " No se pueden modificar el atributo especialista del director <nombre>: alguno de los géneros indicados en la lista no está permitido", según corresponda.

donde <especialista> es la instancia del argumento de especialista que ha provocado que se haya lanzado la excepción y <nombre> es el nombre del director.

6. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: `PrincipalB4E2`

Operaciones a realizar en la función `main()`:

- **a.** Crear la instancia de la película `pelicula2`, indicando `null` en el argumento `nombre`.
- **b.** Crear la instancia de la película `pelicula2`, indicando 2030 en el argumento asociado a `anho`.
- **c.** Mostrar por pantalla las características de `pelicula2`.
- **d.** Crear la instancia de la película `pelicula2`, indicando el año que aparece en el fichero de instancias B3E2_instancias.xlsx para el argumento asociado a `anho`.
- **e.** Mostrar por pantalla las características de `pelicula2`.
- **f.** Cambiar el año de realización de `pelicula2` a 2027 a través del setter `setAnho`.
- **g.** Mostrar por pantalla las características de `pelicula2`.
- **h.** Crear la instancia de la película `pelicula3`, indicando -35 en el argumento asociado a `duracion`.
- **i.** Mostrar por pantalla las características de `pelicula3`.
- **j.** Crear la instancia de la película `pelicula3`, indicando la duración que aparece en el fichero de instancias B3E2_instancias.xlsx para el argumento asociado a `duracion`.
- **k.** Mostrar por pantalla las características de `pelicula3`.
- **l.** Cambiar la duración de realización de `pelicula3` a 4 a través del setter `setAnho`.
- **m.** Mostrar por pantalla las características de `pelicula3`.
- **n.** Crear la instancia del actor `actor3`, indicando `null` en el argumento `nombre`.
- **o.** Mostrar por pantalla las características de `actor3`.
- **p.** Crear la instancia del actor `actor3`, indicando 0 en el argumento asociado a `cache`.
- **q.** Mostrar por pantalla las características de `actor3`.
- **r.** Crear la instancia del actor `actor3`, indicando la cache que aparece en el fichero de instancias B3E2_instancias.xlsx para el argumento asociado a `cache`.
- **s.** Mostrar por pantalla las características de `actor3`.
- **t.** Cambiar la cache del actor `actor3` a -400 a través del setter `setCache`.
- **u.** Mostrar por pantalla las características de `actor3`.
- **v.** Mostrar por pantalla la `importancia` del `actor3` para el tipo de participación de "figurantes".
- **w.** Mostrar por pantalla la `importancia` del `actor3` para el tipo de participación de "secundario".
- **x.** Crear una instancia de `ArrayList<String>`.
- **y.** Cambiar los deseos del actor `actor3` a la instancia creada en el apartado (**x**) a través del setter `setDeseos`.
- **z.** Cambiar los deseos del actor `actor3` a `null` a través del setter `setDeseos`.

aa. Mostrar por pantalla las características de `actor3`.

bb. Crear la instancia de la película `director2`, indicando `null` en el argumento `nombre`.

cc. Mostrar por pantalla las características de `director2`.

dd. Crear la instancia del director `director2` de acuerdo con los valores de los atributos del constructor indicados en el fichero B3E2_instancias.xlsx.

ee. Crear una instancia de `Set<String>`.

ff. Cambiar los valores del atributo `especialista` del director `director2` a la instancia creada en el apartado (**ee**) a través del setter `setEspecialista`.

gg. Cambiar los valores del atributo `especialista` del director `director2` a `null` a través del setter `setEspecialista`.

hh. Mostrar por pantalla las características de `director2`.

ii. Introducir el género "Gore" en la instancia creada en el apartado (**ee**).

jj. Cambiar los valores del atributo `especialista` del director `director2` a la instancia creada en el apartado (**ee**) a través del setter `setEspecialista`.

kk. Mostrar por pantalla las características de `director2`.